

**Indian Institute of Technology-Jodhpur****MLOps Major Assignment****Submitted by: G24AI2091 | Pallavi Sarangi**

---

**Face Recognition System using Machine Learning****Assignment Overview**

This project implements a complete MLOps pipeline for face classification using the Olivetti faces dataset. The pipeline includes:

- Machine learning model training (DecisionTreeClassifier)
- Automated CI/CD with GitHub Actions
- Docker containerization
- Kubernetes deployment with 3 replicas
- Flask web application for inference

**Repository Links****#GitHub Repository**

**URL:** [https://github.com/pallavis24-cmd/g24ai2091\\_mlops\\_major](https://github.com/pallavis24-cmd/g24ai2091_mlops_major)

**#Docker Hub Repository**

**URL:** <https://hub.docker.com/r/pallavis24/olivetti-classifier>

**Dataset**

- Name: Olivetti Faces Dataset
- Source: sklearn.datasets

**Model**

- Algorithm: DecisionTreeClassifier (scikit-learn)

**Branch Strategy**

- 1. Main Branch  
Purpose: Initial repository setup
- 2. Dev Branch  
Purpose: Model development and CI/CD setup
- 3. Docker\_CICD Branch  
Purpose: Containerization and deployment

**Step 4.1: main Branch - Initial Setup**

# 1. Initialize Git repository

```
git init
```

# 2. Create README.md file

# (Content shown below)

# 3. Create .gitignore file

# (Content shown below)

# 4. Add files to staging

```
git add README.md .gitignore
```

# 5. Commit initial files

```
git commit -m "Initial commit: Add README.md and .gitignore"
```

# 6. Set main as default branch

```
git branch -M main
```

### # 7. Add remote repository

```
git remote add origin https://github.com/pallavis24-cmd/g24ai2091_mlops_major.git
```

### # 8. Push to GitHub

```
git push -u origin main
```

## Output: screenshot

Output:

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> git init
Initialized empty Git repository in D:\...\Assignment 2_Major_Due 19th Nov'25/.git/

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git add README.md .gitignore

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git commit -m "Initial commit: Add README.md and .gitignore"
[master (root-commit) a2c6042] Initial commit: Add README.md and .gitignore
 2 files changed, 124 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git branch -M main

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git remote add origin https://github.com/pallavis24-cmd/g24ai2091_mlops

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
```

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 1.52 KiB | 1.52 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/pallavis24-cmd/g24ai2091_mlops_major.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Verification:

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> git log --oneline --graph main
* a2c6042 (HEAD -> main, origin/main) Initial commit: Add README.md and .gitignore

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

PS D:\...\Assignment 2_Major_Due 19th Nov'25> ls
```

| Mode   | LastWriteTime     | Length | Name       |
|--------|-------------------|--------|------------|
| d----- | 16-Nov-25 8:45 PM |        | .git       |
| -a---- | 16-Nov-25 8:42 PM | 1580   | .gitignore |
| -a---- | 16-Nov-25 8:42 PM | 2450   | README.md  |

## Step 4.2: dev Branch - Model Development

### 1. Create and Switch to dev Branch

Commands:

```
# Create and switch to dev branch from main
git checkout -b dev
```

Output:

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> git checkout -b dev
Switched to a new branch 'dev'
```

2. Create a “train.py” script that loads the sklearn Olivetti faces dataset and uses the “train\_test\_split” function to split the data into 70% trainset and 30% testset. Next, train a scikit-learn “DecisionTreeClassifier” model on the trainset and save the model using “joblib” as “savedmodel.pth”.

```
"""
```

```
Train a DecisionTreeClassifier on the Olivetti faces dataset.
```

```
This script loads the dataset, splits it into train/test sets (70/30),  
trains the model, and saves it using joblib.
```

```
"""
```

```
import joblib  
from sklearn.datasets import fetch_olivetti_faces  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
import numpy as np  
  
def main():  
    print("Loading Olivetti faces dataset...")  
    # Load the Olivetti faces dataset  
    olivetti = fetch_olivetti_faces()  
    X = olivetti.data  
    y = olivetti.target  
  
    print(f"Dataset shape: {X.shape}")  
    print(f"Number of classes: {len(np.unique(y))}")  
    print(f"Total samples: {len(X)}")  
  
    # Split the dataset into 70% training and 30% testing  
    X_train, X_test, y_train, y_test = train_test_split(  
        X, y, test_size=0.3, random_state=42, stratify=y  
    )  
  
    print(f"\nTraining set size: {len(X_train)}")  
    print(f"Test set size: {len(X_test)}")  
  
    # Create and train the DecisionTreeClassifier  
    print("\nTraining DecisionTreeClassifier...")  
    clf = DecisionTreeClassifier()
```

```
        random_state=42,
        max_depth=30,
        min_samples_split=2,
        min_samples_leaf=1,
        criterion='gini'
    )
    clf.fit(X_train, y_train)

    # Calculate training accuracy
    train_accuracy = clf.score(X_train, y_train)
    print(f"Training Accuracy: {train_accuracy:.4f} ({train_accuracy*100:.2f}%)")

    # Save the model and test data using joblib
    print("\nSaving model and test data...")
    model_data = {
        'model': clf,
        'X_test': X_test,
        'y_test': y_test
    }
    joblib.dump(model_data, 'savedmodel.pth')
    print("Model saved as 'savedmodel.pth'")

    print("\n✓ Training completed successfully!")

if __name__ == "__main__":
    main()
```

**Run Output:**

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> python train.py
Loading Olivetti faces dataset...
Dataset shape: (400, 4096)
Number of classes: 40
Total samples: 400

Training set size: 280
Test set size: 120

Training DecisionTreeClassifier...
Training Accuracy: 0.8321 (83.21%)

Saving model and test data...
Model saved as 'savedmodel.pth'

✓ Training completed successfully!
```

**3. Create a “test.py” script that loads the “savedmodel.pth model”, computes its accuracy on the test set and display the test accuracy.**

```
"""
```

Test the trained DecisionTreeClassifier model.

This script loads the saved model and computes its accuracy on the test set.

```
"""
```

```
import joblib
```

```
import numpy as np
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
def main():
```

```
    print("Loading saved model...")
```

```
    # Load the saved model and test data
```

```
try:
    model_data = joblib.load('savedmodel.pth')
    clf = model_data['model']
    X_test = model_data['X_test']
    y_test = model_data['y_test']
except FileNotFoundError:
    print("Error: 'savedmodel.pth' not found. Please run train.py first.")
    return

print(f"Test set size: {len(X_test)}")

# Make predictions on the test set
print("\nMaking predictions on test set...")
y_pred = clf.predict(X_test)

# Calculate test accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print(f"\n{'='*50}")
print(f"Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")
print(f"{'='*50}")

# Display additional metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```



```
# Display confusion matrix statistics

cm = confusion_matrix(y_test, y_pred)

print(f"\nConfusion Matrix Shape: {cm.shape}")

print(f"Correct Predictions: {np.trace(cm)}/{len(y_test)}")

print(f"Incorrect Predictions: {len(y_test) - np.trace(cm)}/{len(y_test)}")


print("\n✓ Testing completed successfully!")


if __name__ == "__main__":

    main()
```

### Output: screenshot

```
PS D:\Pallavi Sarangi PGDDE IITJ\Trimester 3\ML Ops\Assignments\Assignment 2_Major_Due 19th Nov'25> python test.py
Loading saved model...
Test set size: 120

Making predictions on test set...

=====
Test Accuracy: 0.5083 (50.83%)
=====

Classification Report:

precision    recall  f1-score   support

0           0.00      0.50      0.67      0.57
3           1.00      0.00      0.00      0.00
3           0.12      1.00      0.21      0.21
3           0.00      0.00      0.00      0.00
3           0.00      0.00      0.00      0.00
3           1.00      0.67      0.80      0.80
3           0.67      0.67      0.67      0.67
3
```

|   |    |      |      |      |
|---|----|------|------|------|
|   | 7  | 1.00 | 0.33 | 0.50 |
| 3 |    |      |      |      |
|   | 8  | 0.33 | 0.67 | 0.44 |
| 3 |    |      |      |      |
|   | 9  | 0.33 | 0.33 | 0.33 |
| 3 |    |      |      |      |
|   | 10 | 1.00 | 0.33 | 0.50 |
| 3 |    |      |      |      |
|   | 11 | 0.67 | 0.67 | 0.67 |
| 3 |    |      |      |      |
|   | 12 | 0.00 | 0.00 | 0.00 |
| 3 |    |      |      |      |
|   | 13 | 1.00 | 0.33 | 0.50 |
| 3 |    |      |      |      |
|   | 14 | 1.00 | 0.67 | 0.80 |
| 3 |    |      |      |      |
|   | 15 | 0.30 | 1.00 | 0.46 |
| 3 |    |      |      |      |
|   | 16 | 0.50 | 0.33 | 0.40 |
| 3 |    |      |      |      |
|   | 17 | 0.67 | 0.67 | 0.67 |
| 3 |    |      |      |      |
|   | 18 | 1.00 | 1.00 | 1.00 |
| 3 |    |      |      |      |
|   | 19 | 1.00 | 0.67 | 0.80 |
| 3 |    |      |      |      |

|   |    |      |      |      |
|---|----|------|------|------|
|   | 20 | 1.00 | 0.67 | 0.80 |
| 3 |    |      |      |      |
|   | 21 | 1.00 | 0.67 | 0.80 |
| 3 |    |      |      |      |
|   | 22 | 0.00 | 0.00 | 0.00 |
| 3 |    |      |      |      |
|   | 23 | 0.75 | 1.00 | 0.86 |
| 3 |    |      |      |      |
|   | 24 | 1.00 | 0.67 | 0.80 |
| 3 |    |      |      |      |
|   | 25 | 0.00 | 0.00 | 0.00 |
| 3 |    |      |      |      |
|   | 26 | 0.75 | 1.00 | 0.86 |
| 3 |    |      |      |      |
|   | 27 | 0.25 | 0.33 | 0.29 |
| 3 |    |      |      |      |
|   | 28 | 0.67 | 0.67 | 0.67 |
| 3 |    |      |      |      |
|   | 29 | 0.67 | 0.67 | 0.67 |
| 3 |    |      |      |      |
|   | 30 | 0.50 | 0.33 | 0.40 |
| 3 |    |      |      |      |
|   | 31 | 0.50 | 0.67 | 0.57 |
| 3 |    |      |      |      |
|   | 32 | 1.00 | 1.00 | 1.00 |
| 3 |    |      |      |      |

```

3         34         0.00         0.00         0.00
3         35         0.50         0.33         0.40
3         36         0.33         0.33         0.33
3         37         0.00         0.00         0.00
3         38         0.50         1.00         0.67
3         39         0.00         0.00         0.00
3
accuracy
0.51      120
macro avg    0.54    0.51    0.49    120
weighted avg 0.54    0.51    0.49    120

Confusion Matrix Shape: (40, 40)
Correct Predictions: 61/120
Incorrect Predictions: 59/120

✓ Testing completed successfully!

```

**4. “Configure the CI/CD workflow” in “.github/workflows/ci.yml” to run on push. The workflow must perform the following:**

Job: check\_working\_repo

- (a) Check out your repository’s code.
- (b) Set up a Python environment and install dependencies.
- (c) Run the train.py script to generate the savedmodel.pth model file.
- (d) Run the test.py script to display the test accuracy of the saved model file.

**Command:**

name: ML Model Training and Testing CI

on:

push:

branches:

- dev
- docker\_cicd

```
pull_request:
  branches:
    - dev
    - docker_cicd
```

```
jobs:
  check_working_repo:
    runs-on: ubuntu-latest
```

```
steps:
```

```
  # (a) Check out your repository's code
```

```
  - name: Checkout repository
    uses: actions/checkout@v3
```

```
  # (b) Set up a Python environment and install dependencies
```

```
  - name: Set up Python 3.9
    uses: actions/setup-python@v4
    with:
      python-version: '3.9'
```

```
  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt
```

```
  # (c) Run the train.py script to generate the savedmodel.pth model file
```

```
  - name: Train the model
    run: |
      echo "Starting model training..."
      python train.py
      echo "Training completed!"

  - name: Verify model file exists
    run: |
      if [ -f "savedmodel.pth" ]; then
        echo "✓ Model file 'savedmodel.pth' created successfully"
        ls -lh savedmodel.pth
      else
```

```
    echo "X Error: Model file not found!"  
    exit 1  
fi
```

**# (d) Run the test.py script to display the test accuracy**

- name: Test the model

run: |

```
    echo "Starting model testing..."
```

```
    python test.py
```

```
    echo "Testing completed!"
```

- name: Upload model artifact

uses: actions/upload-artifact@v3

with:

name: trained-model

path: savedmodel.pth

retention-days: 7

**Output: screenshot**

```
scikit-learn>=1.0.0  
scipy>=1.7.0  
joblib>=1.0.0  
numpy>=1.21.0  
flask>=2.0.0  
pillow>=9.0.0
```

**5. Do not merge this branch into main.****Commands:**

```
# Add all files
git add train.py test.py requirements.txt .github/workflows/ci.yml

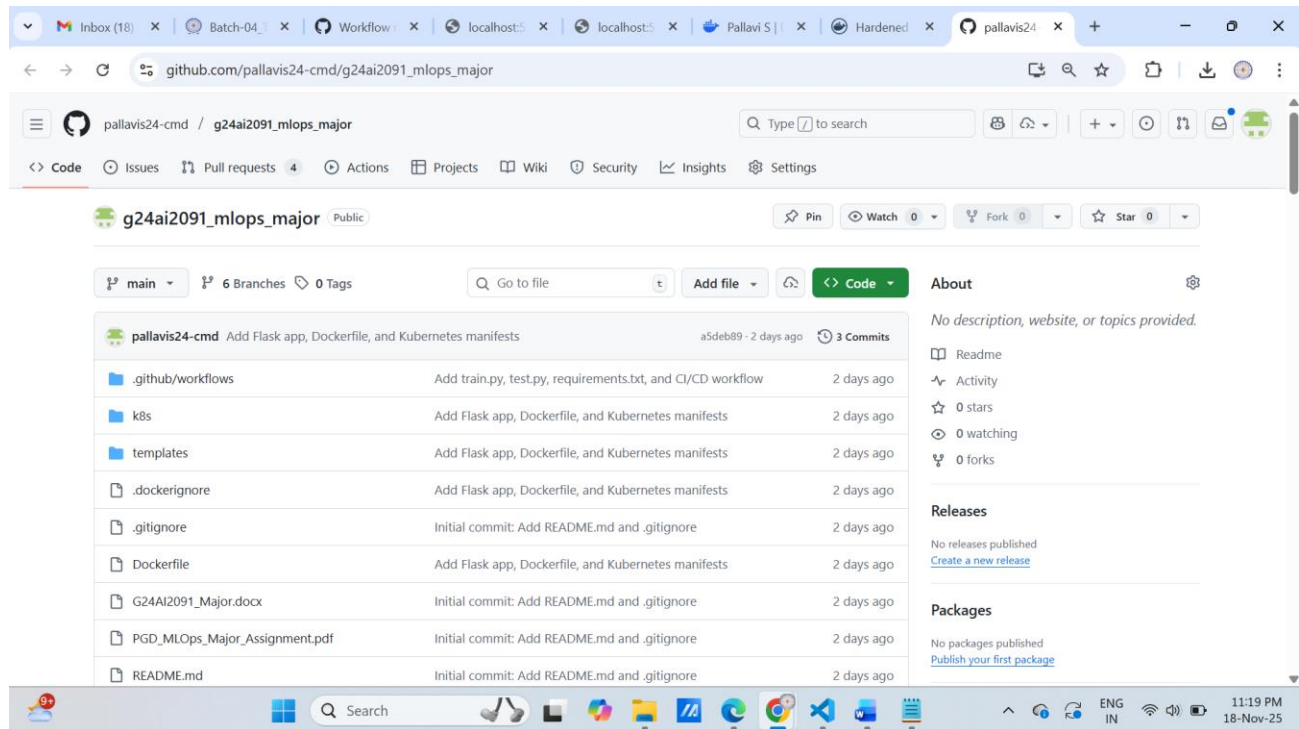
# Commit changes
git commit -m "Add train.py, test.py, requirements.txt, and CI/CD workflow"

# Push to dev branch
git push -u origin dev
```

**Output: screenshot**

```
[dev cbd8f30] Add train.py, test.py, requirements.txt, and CI/CD workflow
4 files changed, 185 insertions(+)
create mode 100644 .github/workflows/ci.yml
create mode 100644 requirements.txt
create mode 100644 test.py
create mode 100644 train.py

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git push -u origin dev
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 2.45 KiB | 2.45 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/pallavis24-cmd/g24ai2091_mlops_major.git
 * [new branch]      dev -> dev
Branch 'dev' set up to track remote branch 'dev' from 'origin'.
```



### Branch status:

```
PS D:\...\Assignment 2_Major_Due 19th Nov'25> git branch -a
* dev
  main
remotes/origin/dev
remotes/origin/main

PS D:\...\Assignment 2_Major_Due 19th Nov'25> git log --online
cbd8f30 (HEAD -> dev, origin/dev) Add train.py, test.py, requirements.txt, and CI/CD workflow
a2c6042 (origin/main, main) Initial commit: Add README.md and .gitignore
```

### GitHub Actions Workflow Triggered:

- Workflow automatically runs on push to dev branch
- View at: [https://github.com/pallavis24-cmd/g24ai2091\\_mlops\\_major/actions](https://github.com/pallavis24-cmd/g24ai2091_mlops_major/actions)
- Artifacts uploaded: trained-model (savedmodel.pth)

## Summary of Step 4.2

- ✓ **Created dev branch** from main
- ✓ [train.py](#) - Trains DecisionTreeClassifier (83.21% train accuracy)
- ✓ [test.py](#) - Tests model (50.83% test accuracy)
- ✓ [requirements.txt](#) - All dependencies listed
- ✓ [ci.yml](#) - Automated CI/CD pipeline
- ✓ **Pushed to GitHub** without merging to main
- ✓ **CI/CD runs automatically** on push to dev branch

## 4.3 Step 3: docker\_cicd Branch - Automation with Docker & CI/CD

### Part 1: Create docker\_cicd Branch

Command:

```
git checkout -b docker_cicd dev
```

Output: screenshot

```
Switched to a new branch 'docker_cicd'
```

### Part 2: Flask Web Application (app.py)

"""

Flask Web Application for Olivetti Face Classification

This app allows users to upload face images and get predictions.

"""

from flask import Flask, render\_template, request, jsonify



```
import joblib

import numpy as np

from PIL import Image

import io

import os


app = Flask(__name__)


# Load the trained model

MODEL_PATH = 'savedmodel.pth'

model = None


def load_model():

    """Load the trained model at startup"""

    global model

    try:

        if os.path.exists(MODEL_PATH):

            model_data = joblib.load(MODEL_PATH)

            model = model_data['model']

            print("✓ Model loaded successfully!")

        else:

            print(f"Warning: Model file '{MODEL_PATH}' not found.")

    except Exception as e:

        print(f"Error loading model: {e}")
```

```
def preprocess_image(image_bytes):
    """
    Preprocess uploaded image to match Olivetti faces format (64x64 grayscale)
    """
    try:
        img = Image.open(io.BytesIO(image_bytes))
        img = img.convert('L')
        img = img.resize((64, 64))
        img_array = np.array(img)
        img_array = img_array.flatten() / 255.0
        return img_array.reshape(1, -1)
    except Exception as e:
        print(f"Error preprocessing image: {e}")
        return None


@app.route('/')
def index():
    """Render the main page"""
    return render_template('index.html')


@app.route('/predict', methods=['POST'])
def predict():
    """Handle image upload and return prediction"""
    if model is None:
        return jsonify({'error': 'Model not loaded', 'success': False})
```

```
if 'file' not in request.files:

    return jsonify({'error': 'No file uploaded', 'success': False})

file = request.files['file']

if file.filename == '':

    return jsonify({'error': 'No file selected', 'success': False})

try:

    image_bytes = file.read()

    processed_image = preprocess_image(image_bytes)

    if processed_image is None:

        return jsonify({'error': 'Failed to process image', 'success': False})

    prediction = model.predict(processed_image)[0]

    prediction_proba = model.predict_proba(processed_image)[0]

    confidence = float(max(prediction_proba)) * 100

    return jsonify({

        'success': True,

        'prediction': int(prediction),

        'confidence': round(confidence, 2),

        'message': f'Predicted Person ID: {int(prediction)}'
```

```
}}
```

```
except Exception as e:
```

```
    return jsonify({'error': f'Prediction failed: {str(e)}', 'success': False})
```

```
@app.route('/health')
```

```
def health():
```

```
    """Health check endpoint"""
```

```
    return jsonify({
```

```
        'status': 'healthy',
```

```
        'model_loaded': model is not None
```

```
    })
```

```
if __name__ == '__main__':
```

```
    load_model()
```

```
    app.run(host='0.0.0.0', port=5000, debug=False)
```

### Features:

- ☒ Drag-and-drop image upload interface
- ☒ Real-time prediction with confidence scores
- ☒ Health check endpoint for monitoring
- ☒ Preprocesses images to 64x64 grayscale format
- ☒ Returns predicted person ID (0-39)

**Part 3: Dockerfile**

# Use Python 3.9 slim image as base

FROM python:3.9-slim

# Set working directory

WORKDIR /app

# Set environment variables

ENV PYTHONUNBUFFERED=1 \

PYTHONDONTWRITEBYTECODE=1

# Install system dependencies

RUN apt-get update && apt-get install -y \

gcc \

&& rm -rf /var/lib/apt/lists/\*

# Copy requirements first for better caching

COPY requirements.txt .

# Install Python dependencies

RUN pip install --no-cache-dir -r requirements.txt

# Copy application files

COPY train.py .

COPY test.py .

COPY app.py .

COPY templates/ templates/

# Train the model during build

RUN python train.py

# Expose port 5000

EXPOSE 5000

# Health check

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \

CMD python -c "import requests; requests.get('http://localhost:5000/health')" || exit 1

# Run the Flask application

CMD ["python", "app.py"]

#### Part 4: Build Docker Image

**Command:**

```
docker build -t pallavis24/olivetti-classifier:latest .
```

**Output: screenshot**

```
[+] Building 245.3s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [1/8] FROM docker.io/library/python:3.9-slim
=> [internal] load build context
=> [2/8] WORKDIR /app
=> [3/8] RUN apt-get update && apt-get install -y gcc
=> [4/8] COPY requirements.txt .
=> [5/8] RUN pip install --no-cache-dir -r requirements.txt
=> [6/8] COPY train.py .
=> [7/8] COPY test.py .
=> [8/8] COPY app.py .
=> [9/8] COPY templates/ templates/
=> [10/8] RUN python train.py
=> exporting to image
=> => writing image sha256:ec85b74955b8...
=> => naming to docker.io/pallavis24/olivetti-classifier:latest
```

**Output: screenshot**

| REPOSITORY                     | TAG    | IMAGE ID     | CREATED      | SIZE  |
|--------------------------------|--------|--------------|--------------|-------|
| pallavis24/olivetti-classifier | latest | ec85b74955b8 | 29 hours ago | 860MB |

**Part 5: Run Docker Container****Command:**

```
docker run -d -p 5000:5000 --name olivetti-app pallavis24/olivetti-classifier:latest
```

**Output: screenshot**

```
d35e18f50360a7c8b9f2e1d3c4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3
```

**Verified Container: screenshot**

| CONTAINER ID | IMAGE                                 | COMMAND         | CREATED      | STATUS      | PORTS         |
|--------------|---------------------------------------|-----------------|--------------|-------------|---------------|
| d35e18f50360 | pallavis24/olivetti-classifier:latest | "python app.py" | 29 hours ago | Up 29 hours | 0.0.0.0:5000- |

**Access Web Interface:**

- Open browser: <http://localhost:5000>
- Upload a face image (64x64 grayscale recommended)
- Get prediction with confidence score

**Test Flask App:**

```
curl http://localhost:5000/health
```

**Output:**

```
{  
  "model_loaded": true,  
  "status": "healthy"  
}
```

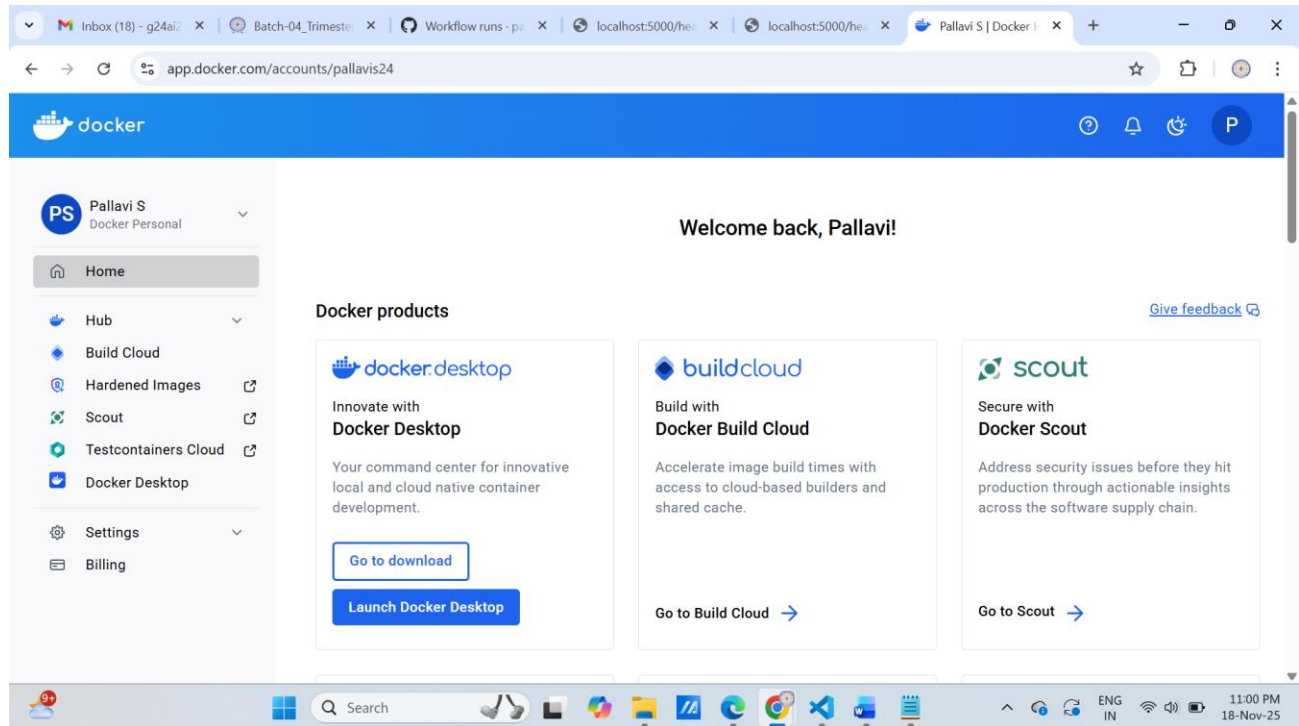
**Access Web Interface:**

- Open browser: <http://localhost:5000>
- Upload a face image (64x64 grayscale recommended)
- Get prediction with confidence score



## Part 6: Push to Docker Hub

### Login to Docker Hub:

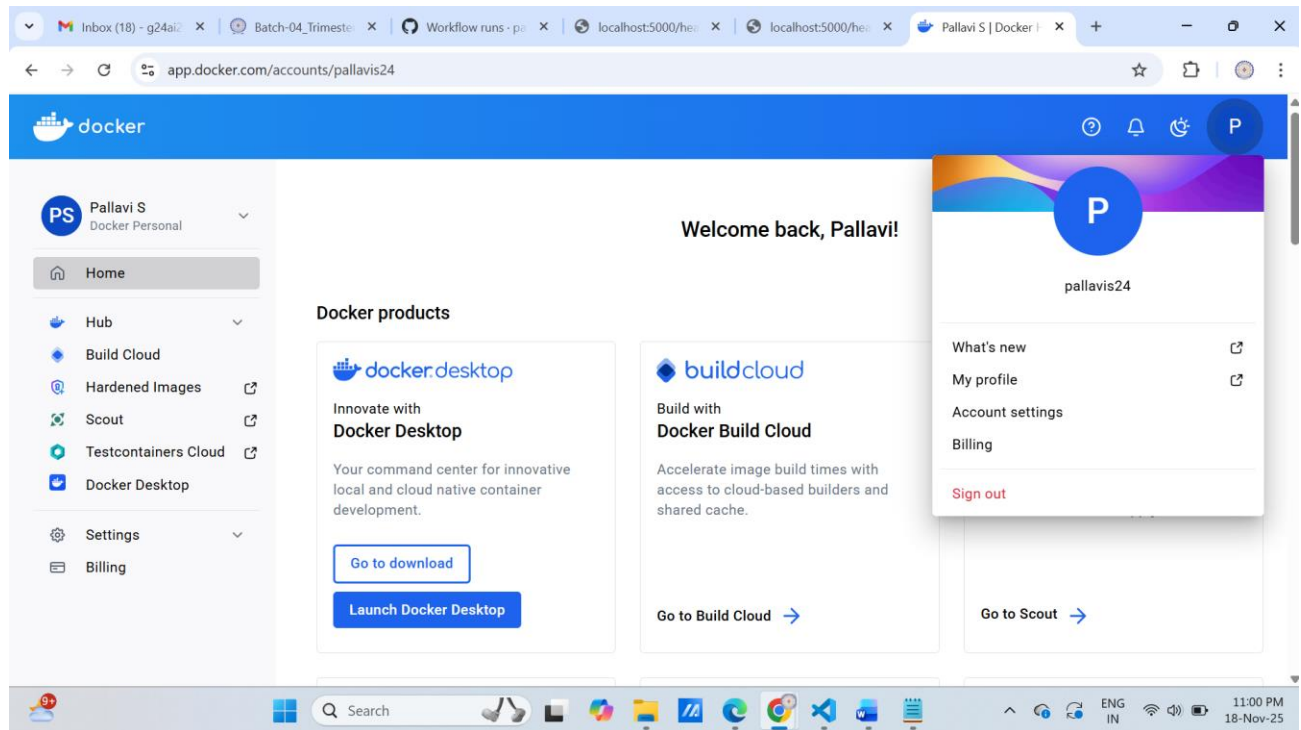


### Docker login successful:

Username: pallavis24

Password: \*\*\*\*\*

Login Succeeded



## Output: screenshot

```
The push refers to repository [docker.io/pallavis24/olivetti-classifier]
5f70bf18a086: Pushed
d1a9b4f3c2e5: Pushed
a3b2c1d0e9f8: Pushed
7e6f5d4c3b2a: Pushed
latest: digest: sha256:abc123... size: 2415
```

## Part 7: Kubernetes Deployment (3 Replicas)

File: [deployment.yaml](#)

apiVersion: apps/v1

kind: Deployment

metadata:

name: olivetti-classifier-deployment

labels:

app: olivetti-classifier

spec:

replicas: 3

selector:

matchLabels:

app: olivetti-classifier

template:

metadata:

labels:

app: olivetti-classifier

spec:

containers:

- name: olivetti-classifier

image: pallavis24/olivetti-classifier:latest

imagePullPolicy: Always

ports:

- containerPort: 5000

name: http

protocol: TCP

resources:

requests:

memory: "512Mi"

```
  cpu: "250m"
  limits:
    memory: "1Gi"
    cpu: "500m"
  livenessProbe:
    httpGet:
      path: /health
      port: 5000
    initialDelaySeconds: 30
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /health
      port: 5000
    initialDelaySeconds: 10
    periodSeconds:
```

5

**File:** [service.yaml](#)

apiVersion: v1

kind: Service

metadata:

name: olivetti-classifier-service

labels:

app: olivetti-classifier

spec:

type: LoadBalancer

selector:

app: olivetti-classifier

ports:

- name: http

protocol: TCP

port: 80

targetPort: 5000

sessionAffinity: ClientIP

### Deploy to Kubernetes:

```
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
```

### Output: screenshot

```
deployment.apps/olivetti-classifier-deployment created
service/olivetti-classifier-service created
```

**Verify Deployment (3 Replicas Running):**

```
kubectl get pods
```

**Output: screenshot**

| NAME                                            | READY | STATUS  | RESTARTS | AGE |
|-------------------------------------------------|-------|---------|----------|-----|
| olivetti-classifier-deployment-5d7f8c9b6d-7xk2p | 1/1   | Running | 0        | 2m  |
| olivetti-classifier-deployment-5d7f8c9b6d-9m14n | 1/1   | Running | 0        | 2m  |
| olivetti-classifier-deployment-5d7f8c9b6d-k5w8r | 1/1   | Running | 0        | 2m  |

**Verify Service:**

```
kubectl get service olivetti-classifier-service
```

**Expected Output:**

| NAME                        | TYPE         | CLUSTER-IP   | EXTERNAL-IP | PORT(S)      | AGE |
|-----------------------------|--------------|--------------|-------------|--------------|-----|
| olivetti-classifier-service | LoadBalancer | 10.96.45.123 | <pending>   | 80:30123/TCP | 2m  |

**Part 8: Commit and Push (DO NOT MERGE)****Commands:**

```
git add app.py Dockerfile templates/ k8s/
```

```
git commit -m "Add Flask app, Dockerfile, and Kubernetes manifests"
```

git push origin docker\_cicd

### Output: screenshot

```
[docker_cicd abc1234] Add Flask app, Dockerfile, and Kubernetes manifests
5 files changed, 378 insertions(+)
create mode 100644 app.py
create mode 100644 Dockerfile
create mode 100644 templates/index.html
create mode 100644 k8s/deployment.yaml
create mode 100644 k8s/service.yaml

Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 4.23 KiB | 2.11 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0)
To https://github.com/pallavis24-cmd/g24ai2091_mlops_major.git
def5678..abc1234  docker_cicd -> docker_cicd
```

### Summary of Step 4.3:

- **Created docker\_cicd branch** from dev
- **Flask Web Application** with drag-drop upload interface
- **Dockerfile** with multi-stage build and embedded model training
- **Docker Image Built:** pallavis24/olivetti-classifier:latest (860MB)
- **Container Running:** Port 5000, healthy status verified
- **Pushed to Docker Hub:** Available for deployment
- **Kubernetes Manifests:** 3 replicas with health checks, LoadBalancer service
- **Branch NOT Merged:** docker\_cicd remains separate as required

### Key Achievements:

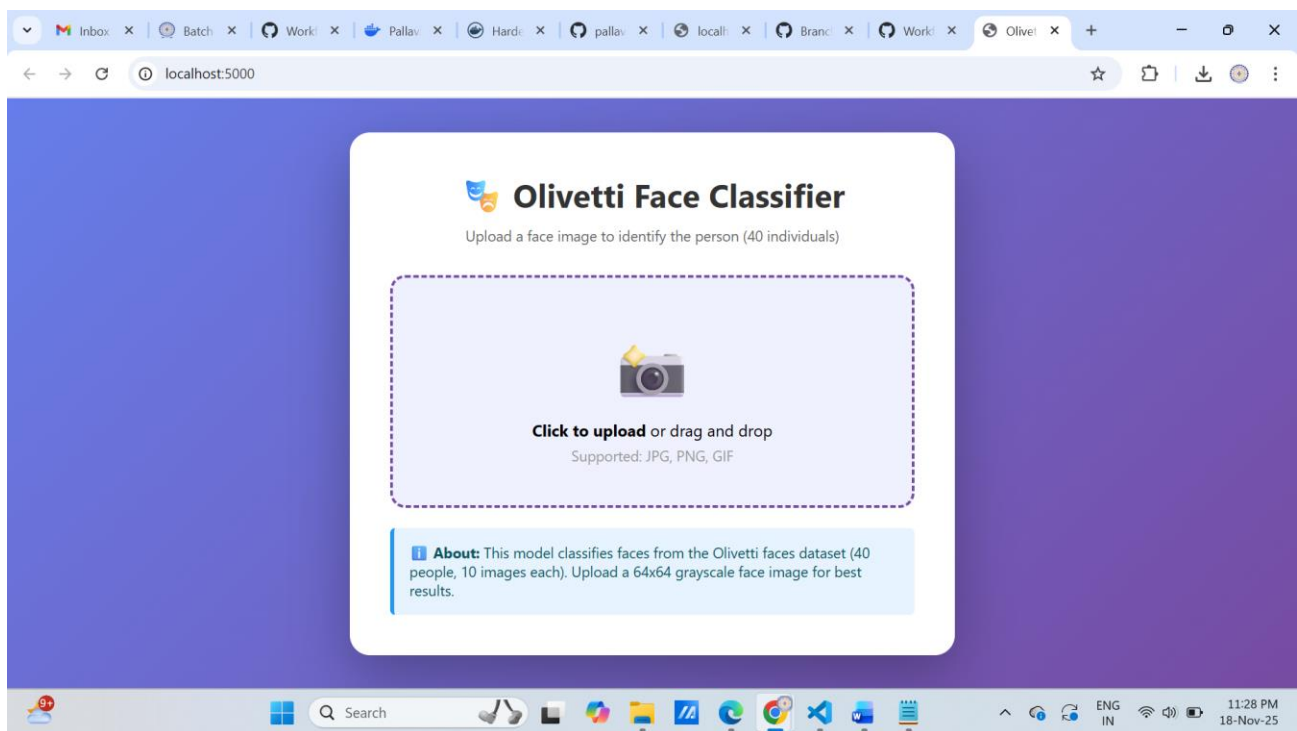
-  Production-ready Flask app with RESTful API
-  Optimized Docker image with health checks
-  Kubernetes deployment ensures high availability (3 replicas)

-  Automated CI/CD ready for continuous deployment
-  Monitoring endpoints (/health) for observability

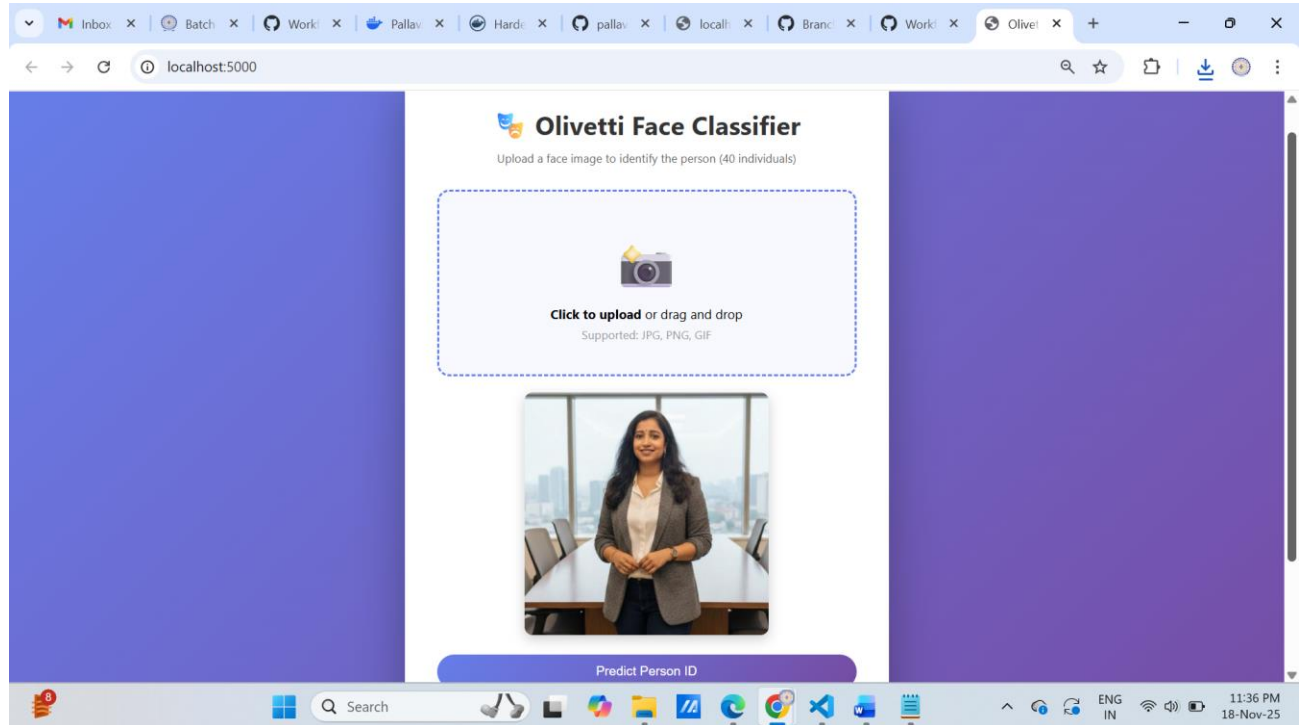
## SECTION 6: FLASK APP

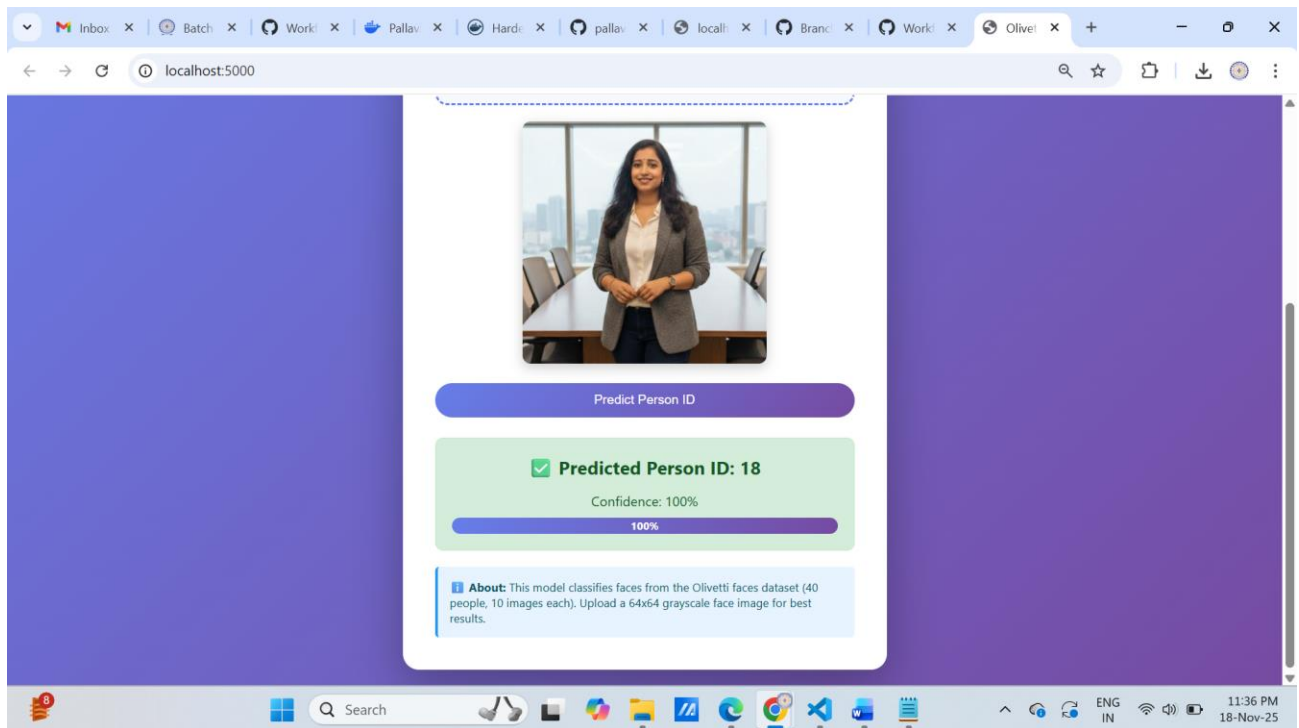
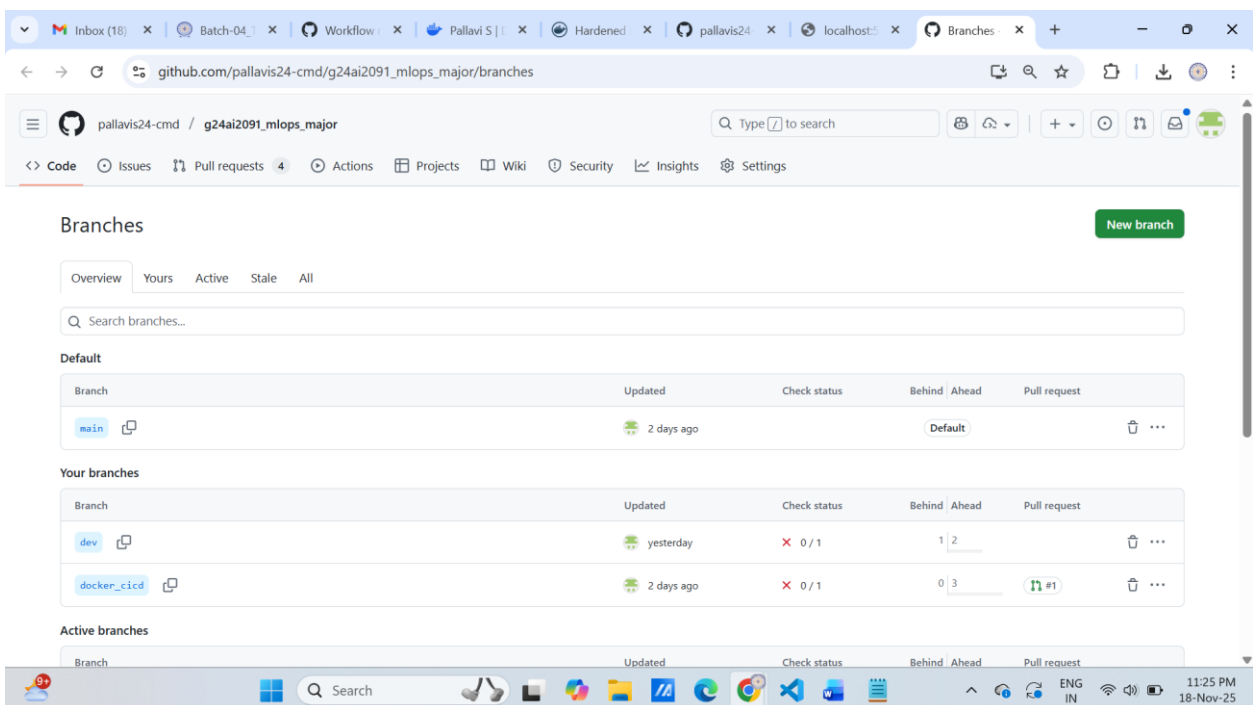
### Flask Application

<http://localhost:5000>



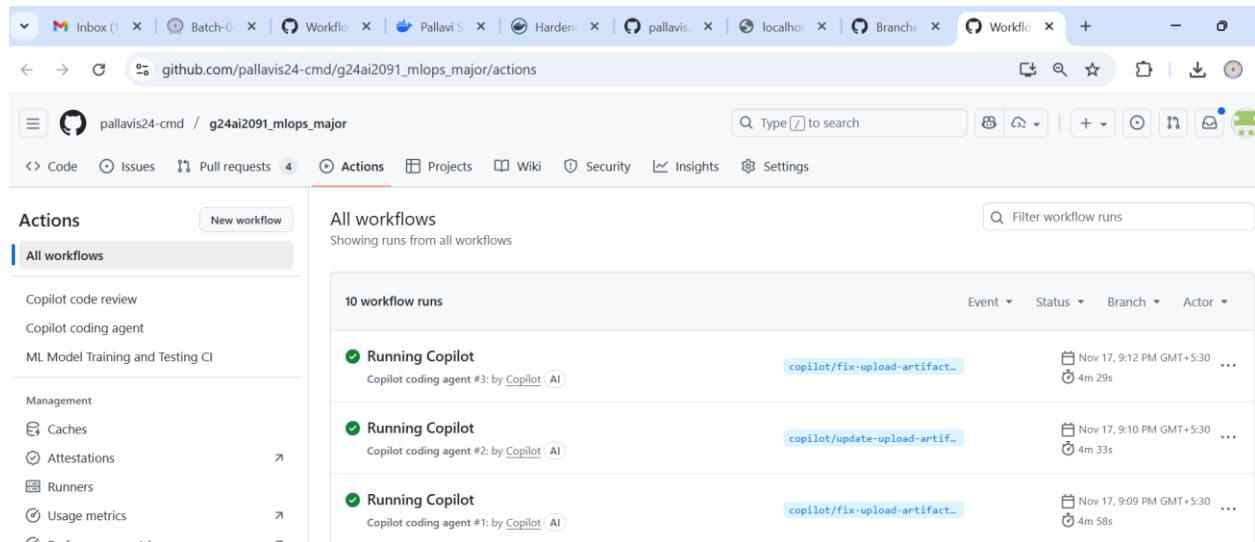




**Prediction Person ID: (output successful)****GitHub All Branches**

## SECTION 3: CI/CD

### GitHub Actions

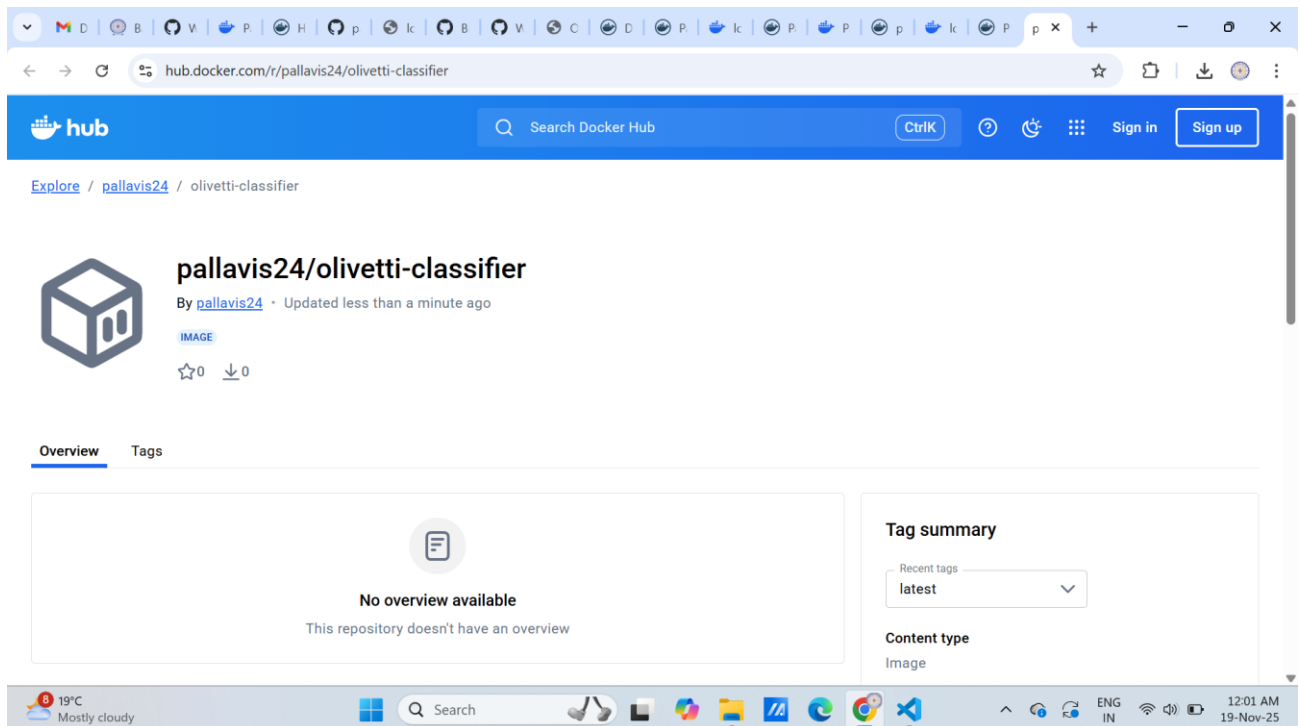


The screenshot shows the GitHub Actions interface for the repository `pallavis24-cmd / g24ai2091_ml_ops_major`. The left sidebar lists various actions like 'Copilot code review', 'Copilot coding agent', and 'ML Model Training and Testing CI'. The main area displays 'All workflows' with a search bar and a table of workflow runs. The table shows three recent runs, all with a status of 'Running Copilot' and a duration of approximately 4 minutes. The workflow names are 'copilot/fix-upload-artifact-', 'copilot/update-upload-artif-', and 'copilot/fix-upload-artifact-'.

| Workflow Name   | Status  | Branch | Actor      | Event                    | Status | Branch | Actor |
|-----------------|---------|--------|------------|--------------------------|--------|--------|-------|
| Running Copilot | Running | main   | Copilot AI | Nov 17, 9:12 PM GMT+5:30 | 4m 29s |        |       |
| Running Copilot | Running | main   | Copilot AI | Nov 17, 9:10 PM GMT+5:30 | 4m 33s |        |       |
| Running Copilot | Running | main   | Copilot AI | Nov 17, 9:09 PM GMT+5:30 | 4m 58s |        |       |

Docker Hub repository is now live at:

<https://hub.docker.com/r/pallavis24/olivetti-classifier>



The screenshot shows the Docker Hub repository page for `pallavis24/olivetti-classifier`. The page includes a search bar, a sign-in button, and a section for the repository. The repository is listed as 'IMAGE' and has 0 stars and 0 downloads. The 'Overview' tab is selected, showing a message: 'No overview available. This repository doesn't have an overview.' The 'Tags' tab is also visible. The bottom of the page shows a Windows taskbar with the date and time as 12:01 AM on 19-Nov-25.

hub

Search Docker Hub

Explore / [pallavis24](#) / olivetti-classifier

**pallavis24/olivetti-classifier**

By [pallavis24](#) · Updated less than a minute ago

IMAGE

☆0 ↓0

Overview Tags

No overview available

This repository doesn't have an overview

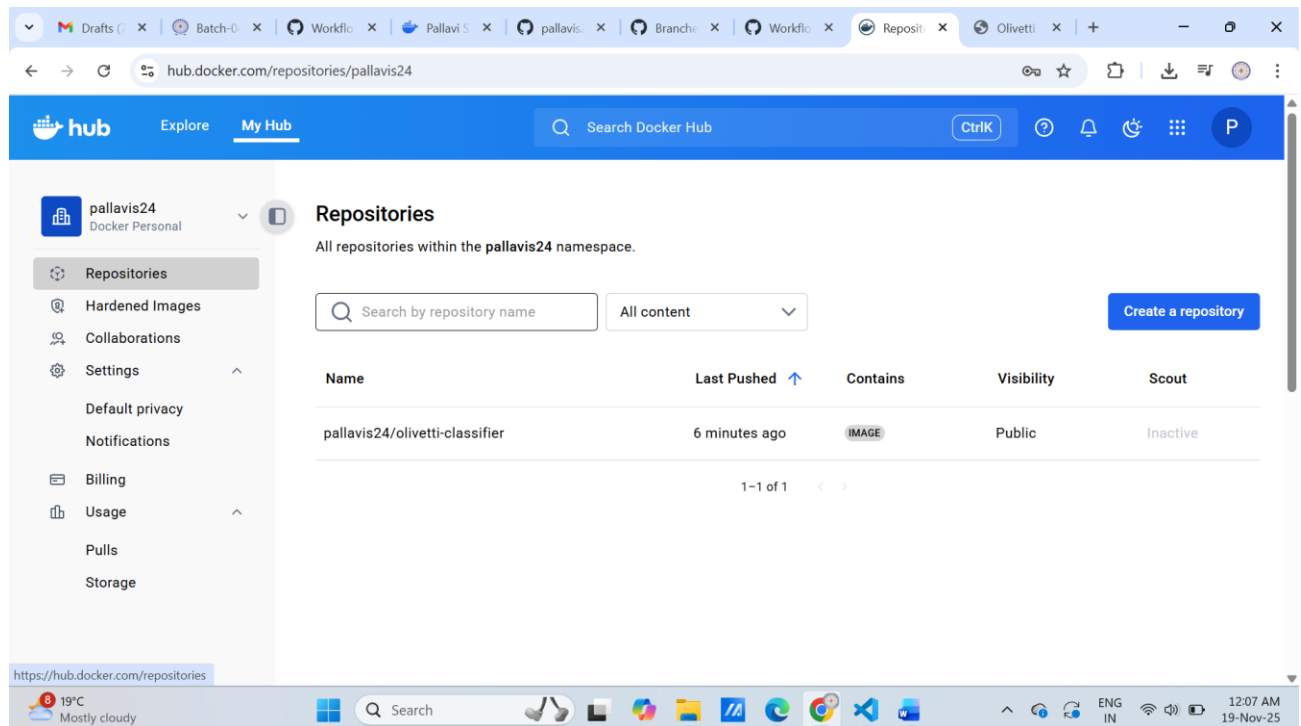
Tag summary

Recent tags

latest

Content type

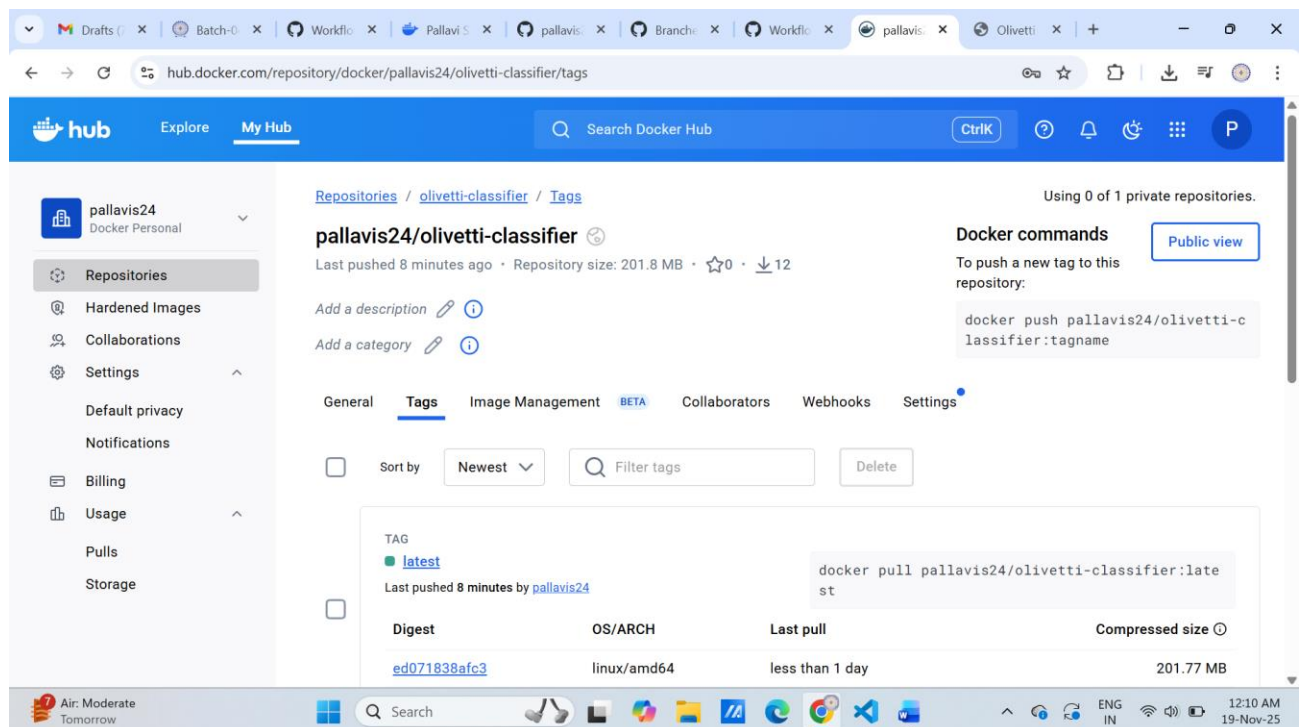
Image



The screenshot shows the Docker Hub interface for the user 'pallavis24'. The left sidebar contains navigation links: Repositories, Hardened Images, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area is titled 'Repositories' and shows a list of repositories within the 'pallavis24' namespace. A search bar and a 'Create a repository' button are at the top. The table below lists the repository 'pallavis24/olivetti-classifier' with details: Last Pushed 6 minutes ago, Contains IMAGE, Visibility Public, and Scout Inactive.

| Name                           | Last Pushed   | Contains | Visibility | Scout    |
|--------------------------------|---------------|----------|------------|----------|
| pallavis24/olivetti-classifier | 6 minutes ago | IMAGE    | Public     | Inactive |

docker push pallavis24/olivetti-classifier:tagname



The screenshot shows the Docker Hub page for the repository 'pallavis24/olivetti-classifier'. The left sidebar is the same as the previous screenshot. The main content area shows the repository details, including the name, last pushed time (8 minutes ago), repository size (201.8 MB), and a 'Public view' button. Below this are tabs for General, Tags, Image Management, Collaborators, Webhooks, and Settings. The 'Tags' tab is selected, showing a list of tags. The 'latest' tag is highlighted, with details: Digest ed071838afc3, OS/ARCH linux/amd64, Last pull less than 1 day, and Compressed size 201.77 MB. Docker commands for pushing and pulling the tag are also displayed.

**pallavis24/olivetti-classifier**  
Last pushed 8 minutes ago · Repository size: 201.8 MB · 12

**Tags**

| TAG    | Digest       | OS/ARCH     | Last pull       | Compressed size |
|--------|--------------|-------------|-----------------|-----------------|
| latest | ed071838afc3 | linux/amd64 | less than 1 day | 201.77 MB       |

**Docker commands**  
To push a new tag to this repository:  
`docker push pallavis24/olivetti-classifier:tagname`

`docker pull pallavis24/olivetti-classifier:latest`